



Inspiring Excellence

CSE370 : Database Systems

Project Report

Project Title : BSC_Support_Hub

Group No : 11, CSE370 Lab Section : 01, Summer 2025		
ID	Name	Contribution
22201783	Abdullah Al Muqit	Registration, login, logout, Resources and Mentorship
22301520	Gobinda Ghosh	Jobs , Announcements and Profile update

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	4
3	ER/EER Diagram	5
4	Schema Diagram	6
5	Normalization	7-9
6	Frontend Development	10-17
7	Backend Development	18-27
8	Source Code Repository	28
9	Conclusion	29
10	References	29

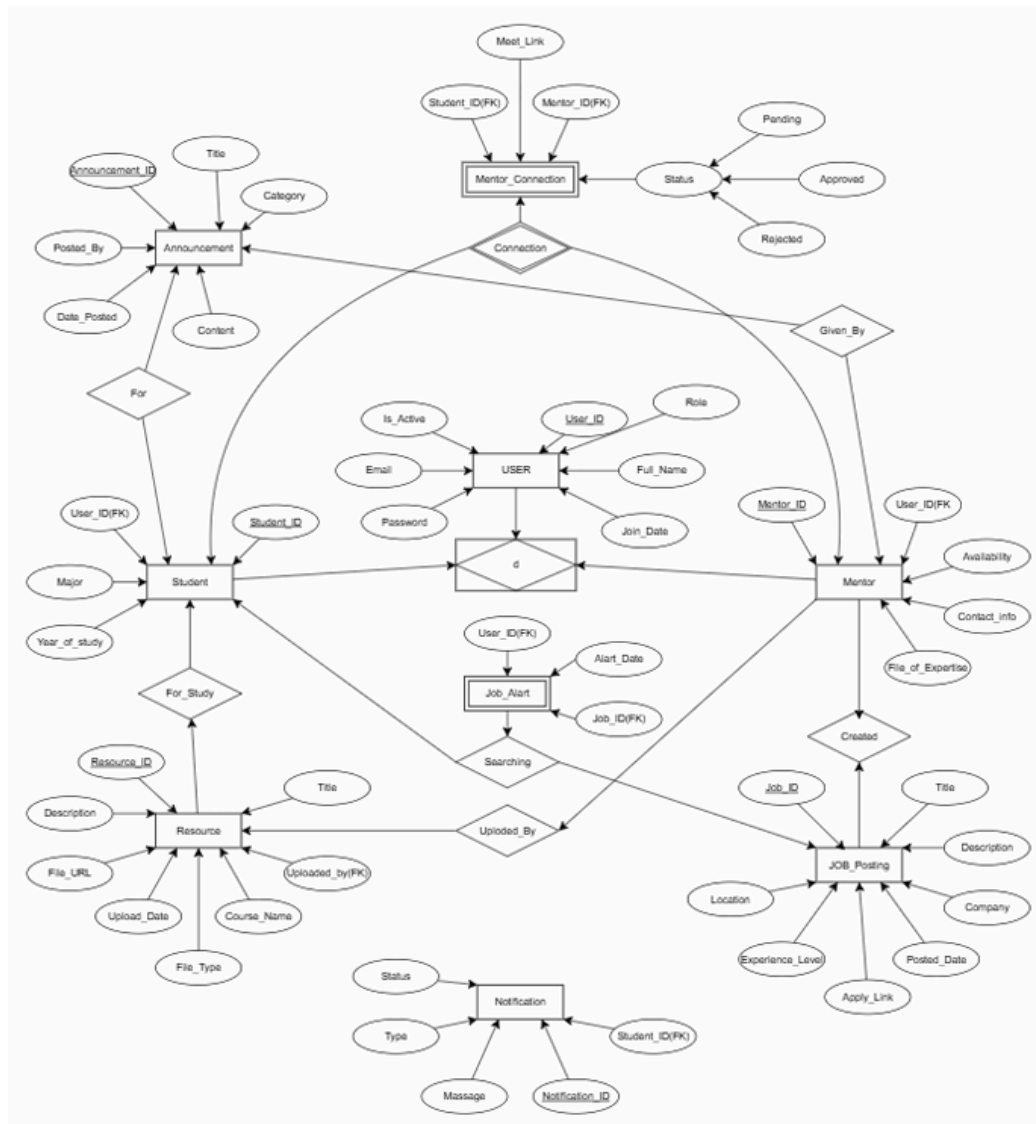
Introduction

The BSC Support Hub is a web platform designed to help undergraduate students in their academic and career journey. It brings together different types of support in one place, such as study resources, mentorship, job and internship opportunities, announcements, and wellness tools. The system allows students to access daily and weekly resources, connect with mentors, apply for jobs, and stay updated with important notices. It also has features like a personalized dashboard, skills tracking, and feedback options to improve student experience. With categories, announcements, and user roles clearly organized, the platform ensures easy navigation and accessibility. Overall, the BSC Support Hub aims to create a student-centered environment that supports learning, growth, and well-being in a simple and effective way.

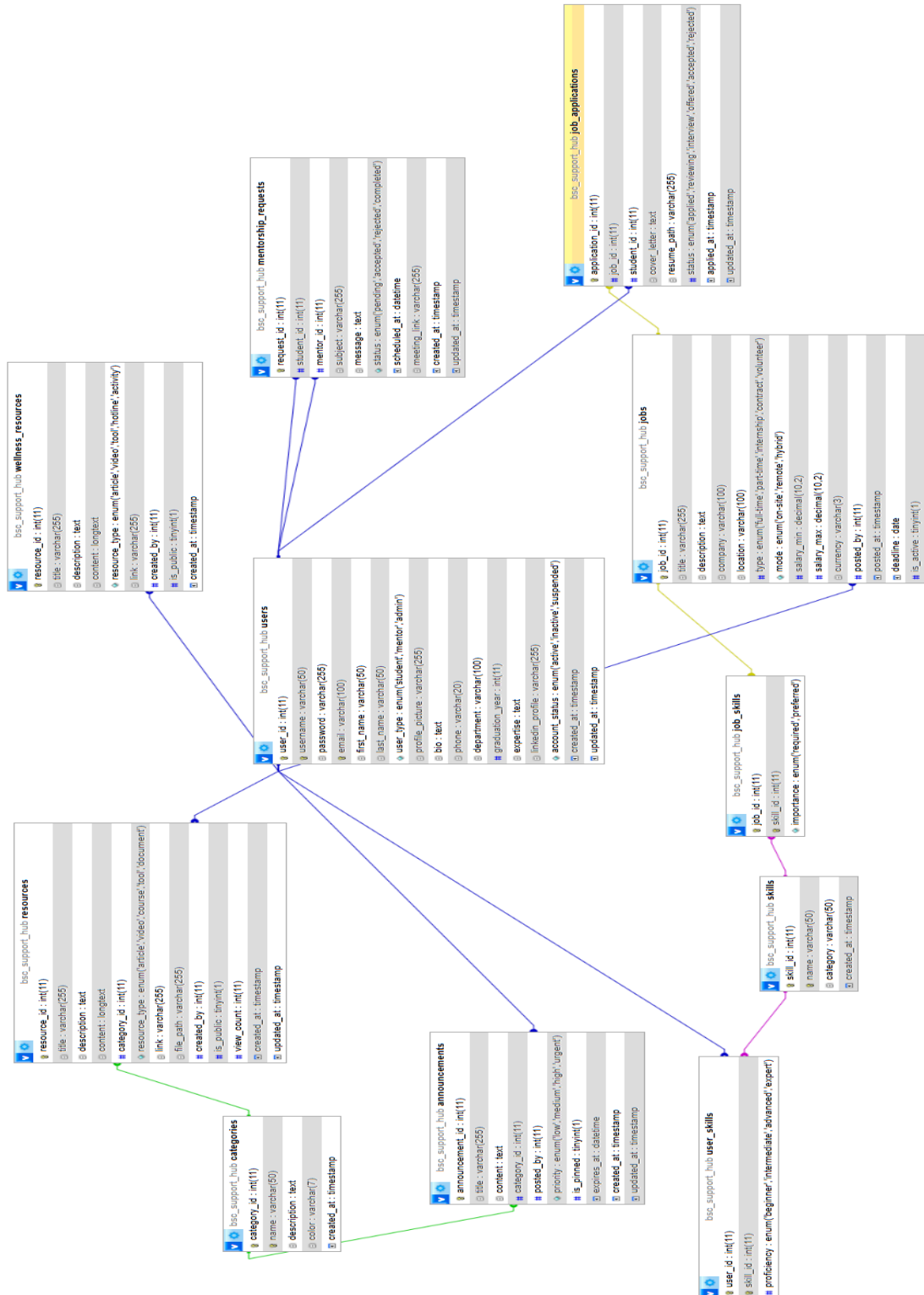
Project Features

ID, Name	Features [3 per member]	
22201783, Abdullah Al Muqit	Ft 1	Registration,login & logout
	Ft 2	Resources
	Ft 3	Mentorship
22301520, Gobinda Ghosh	Ft 1	Jobs
	Ft 2	Announcements
	Ft 3	Profile update

ER/EER Diagram



Schema Diagram



Normalization

- a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
- b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
- c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

a. First Normal Form (1NF):

Rule:

- Each cell must hold atomic values (no repeating groups, no multi-valued attributes).
- Each record must be unique.

Checking on my schema:

- Most tables look atomic. For example: Users(Username, Email, Mobile, ID) are atomic.
- Potential problems:
 1. The users table has an expertise field (TEXT) that may contain multiple skills (e.g., "Web Development, JavaScript, React"), which is a multi-valued attribute.
 2. The categories table has a color field that might be used to store multiple color values.

Solution (Decompose to 1NF):

1. Break repeating groups into separate relations:
2. Remove expertise from users table since skills are already properly stored in the user_skills junction table.
3. For categories, the color field is acceptable as a single value representing a category color, so no decomposition needed.

After decomposition, the schema is in 1NF.

b. Second Normal Form (2NF):

Rule:

- Must be in 1NF.
- No partial dependency (i.e., no non-key attribute should depend on only part of a composite primary key).

Check on your schema:

- Tables with composite primary keys:
 1. user_skills(user_id, skill_id) with non-key attribute proficiency
 2. job_skills(job_id, skill_id) with non-key attribute importance
 3. study_group_members(group_id, user_id) with non-key attributes role and joined_at
 4. event_registrations(event_id, user_id) with non-key attributes registered_at and attended
 5. resource_reviews(resource_id, user_id) with non-key attributes rating and comment
- Possible partial dependencies:
 1. In user_skills, proficiency depends on both user_id and skill_id (the whole key), so no partial dependency.
 2. In job_skills, importance depends on both job_id and skill_id (the whole key), so no partial dependency.
 3. In study_group_members, role and joined_at depend on both group_id and user_id (the whole key), so no partial dependency.
 4. In event_registrations, registered_at and attended depend on both event_id and user_id (the whole key), so no partial dependency.
 5. In resource_reviews, rating and comment depend on both resource_id and user_id (the whole key), so no partial dependency.

Solution (Decompose to 2NF):

- No decomposition needed as all non-key attributes depend on the entire composite key.

After analysis, the schema is in 2NF.

c. Third Normal Form (3NF):

Rule:

- Must be in 2NF.
- No transitive dependency (non-key attribute depending on another non-key attribute).

Checking on my schema:

- Possible transitive dependencies:
 1. In users table, department is a string that might be transitively dependent if we had a separate departments table. Currently, it depends directly on user_id.
 2. In resources table, resource_type is an ENUM that might be transitively dependent if we had a separate resource_types table. Currently, it depends directly on resource_id.
 3. In jobs table, type and mode are ENUMs that might be transitively dependent if we had separate job_types and job_modes tables. Currently, they depend directly on job_id.
 4. In skills table, category is a string that might be transitively dependent if we had a separate skill_categories table. Currently, it depends directly on skill_id.

Final Result:

- The original schema had one 1NF violation (multi-valued expertise field in users table).
- After removing expertise, the schema is in 1NF.
- The schema is in 2NF because there are no partial dependencies in tables with composite keys.
- After decomposing ENUM fields into separate tables, the schema is in 3NF with no transitive dependencies.

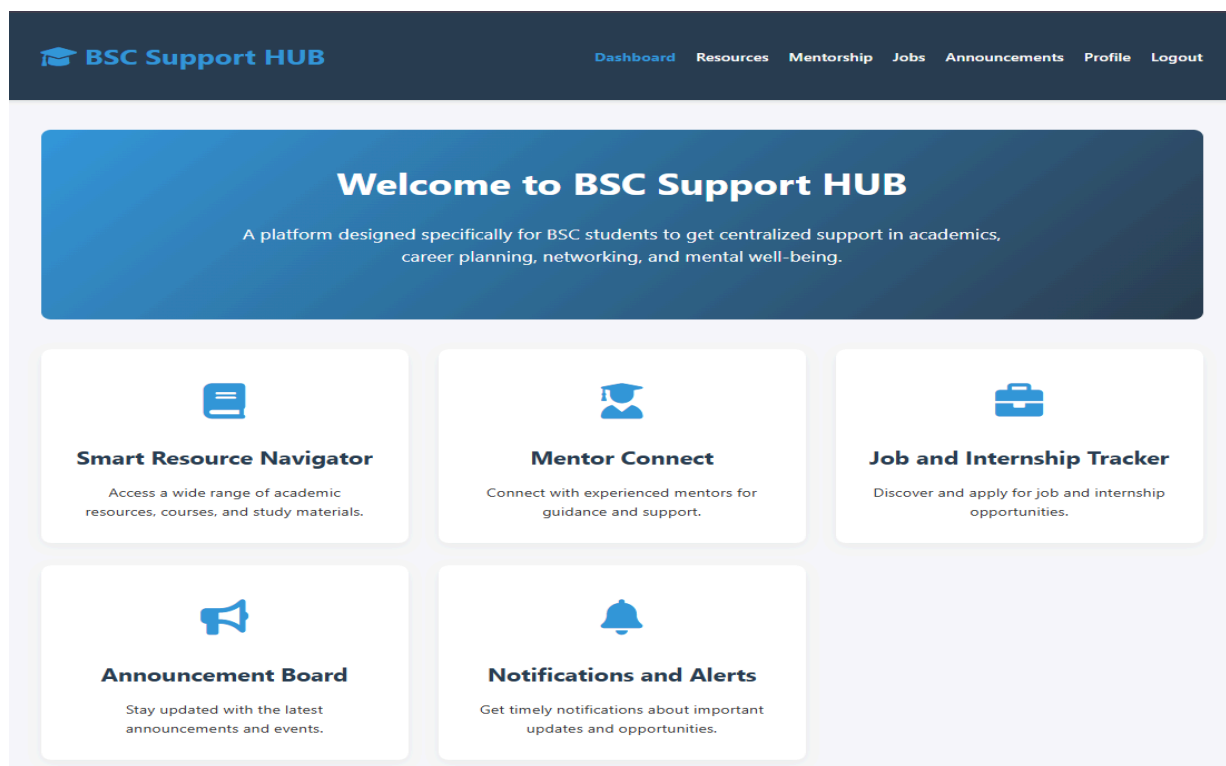
This normalization ensures data integrity, reduces redundancy, and improves the overall design of the BSC Support Hub database.

Frontend Development

Firstly, there is the homepage of our BSC Support Hub platform. For front-end development, we used HTML (Hypertext Markup Language) and CSS (Cascading Style Sheets) to create the structure and design. HTML provides the basic framework of the website, while CSS is used to style and make the pages visually appealing. The website includes several layers and pages such as the main dashboard, mentor connect page, job and internship tracker, announcement board, and personalized dashboard. Additionally, there are specific pages like the resource navigator, event reminder page, and the registration page. Each section is designed to ensure smooth navigation and user interaction. By using these technologies, we've created a user-friendly interface that helps students easily access academic resources, connect with mentors and track career opportunities.

Contribution of ID : 22201783, Name : Abdullah Al Muqit

Deshboard: This homepage provides a central portal for BSC students to access resources, find mentors, track jobs, view announcements, and receive notifications.



Resources: This page displays a searchable and filterable list of academic resources, allowing users to browse and view various materials posted by mentors and students

Post Resource

Category:

All Categories

Search:

FilterReset

Debug Info: Found 6 resources.

Tool

gd x xfd hgc

Wellness

grsfrynv w6 s t

0 viewsby al muqit

View Resource

Course

Project Alpha

Networking

hasbjhdcldb gdcbsdch vdg vbdcfdsdnh
bhjsdvsdvs chdfvrwcvc dcbxchbv

0 viewsby al muqit

View Resource

Course

Introduction to Computer Science

Academics

Basic concepts of computer science

0 viewsby John Doe

View Resource

Course

Data Structures and Algorithms

Academics

Learn about fundamental data structures and algorithms

0 viewsby John Doe

View Resource

Article

Career Guide for CS Students

Career

A comprehensive guide for computer science students

0 viewsby Jane Smith

View Resource

Article

Stress Management Techniques

Wellness

Effective strategies for managing academic stress

0 viewsby John Doe

View Resource

Registrations: This page shows a registration form where new users can create an account by entering their details and choosing their role as student or mentor.

Create an Account

Username

Email

First Name

Last Name

I am a

Student ▼

Password

Confirm Password

[Register](#)

Already have an account? [Login here](#)

Mentorship: This page allows mentors to view, accept, or reject mentorship requests from students, displaying each request's message, date, and current status.

Mentor Connect	
Manage mentorship requests from students.	
Mentorship Requests	
CSS Message: vadxxsidfcss Date: September 7, 2025, 9:41 pm AcceptReject	Pending
hope Message: zawcxvbtuexrc edrftg Date: September 7, 2025, 11:57 am	Rejected
final Message: dedbuhmnbsbvncvbnmi Date: September 7, 2025, 11:54 am	Accepted

Contribution of ID :22301520, Name : Gobinda Ghosh

Jobs : This page lets mentors post, filter, edit, or delete job and internship listings, showing each role's details, salary, deadline, and type or mode.

Post Job

Type:
All Types

Mode:
All Modes

Filter

Reset

vhbsfkvdj vbalvbv vbkds

Internship Hybrid

xcsvdf China

hncfcofcio...

\$3.00 - \$6.00

Deadline: Feb 7, 2025

Edit

Delete

Ai Photo

Internship Remote

LAMA UK

ad hnyhtqmmenh fbnk yu...

\$1.00 - \$3.00

Deadline: Dec 27, 2025

Edit

Delete

Software Developer Intern

Internship Hybrid

Tech Corp San Francisco, CA

Internship position for aspiring software developers...

\$20.00 - \$25.00

Deadline: Dec 31, 2023

Data Analyst

Full-time Remote

Data Inc New York, NY

Full-time position for data analysts...

\$60,000.00 - \$80,000.00

Deadline: Nov 30, 2023

Job and Internship Tracker

Discover and apply for job and internship opportunities.

Type:
All Types

Mode:
All Modes

Filter

Reset

vhbsfkvdj vbalvbv vbkds

Internship Hybrid

xcsvdf China

hncfcofcio...

\$3.00 - \$6.00

Deadline: Feb 7, 2025

Applied

Ai Photo

Internship Remote

LAMA UK

ad hnyhtqmmenh fbnk yu...

\$1.00 - \$3.00

Deadline: Dec 27, 2025

Applied

Software Developer Intern

Internship Hybrid

Tech Corp San Francisco, CA

Internship position for aspiring software developers...

\$20.00 - \$25.00

Deadline: Dec 31, 2023

Applied

Data Analyst

Full-time Remote

Data Inc New York, NY

Full-time position for data analysts...

\$60,000.00 - \$80,000.00

Deadline: Nov 30, 2023

Apply Now

14

Profile update: This page allows a mentor to view and update their profile details, including name, email, bio, department, expertise, and LinkedIn profile



zbt zt
Student
User ID: 10

First Name:

Last Name:

Email:

Bio:

Phone:

Department:

Graduation Year:

your information and preferences.



GG Mentor

Mentor

User ID: 8

First Name:

GG

Last Name:

Mentor

Email:

gg@mentor.net

Bio:

Enter your bio

Phone:

Add phone number

Department:

Enter department

Expertise:

Add expertise

LinkedIn Profile:

Paste LinkedIn URL

Update Profile

Announcements: This page displays important announcements, allowing mentors to post, pin, edit, or delete updates, with category, priority, and status clearly shown.

Post Announcement

Welcome to BSC Support Hub

NetworkingHighPinned

Welcome to our new platform! Here you can find resources, connect with mentors, and discover job opportunities.

Posted by John DoeSeptember 5, 2025, 9:53 pm

Final Exam Schedule Released

AcademicsHighPinned

The final exam schedule for this semester has been released. Please check your student portal.

Posted by John DoeSeptember 5, 2025, 9:53 pm

sgdbznbgf

NetworkingMediumPinned

sgdvfd ngnf

Posted by al muqitSeptember 7, 2025, 12:03 pmExpires: Sep 12, 2025, 12:02 am

EditDelete

Backend Development

Contribution of ID : 22201783, Name : Abdullah Al Muqit

Registration:

This code is for user registration, allowing new users to sign up by entering their details on the platform.

- It checks if all fields are filled, passwords match, and whether the username or email already exists.
- If validation passes, it hashes the password, saves the new user to the database, and redirects to the login page.

```
// Validate inputs
if(empty($username) || empty($email) || empty($first_name) || empty($last_name) || empty($password) || empty($confirm_password)) {
    $error = 'All fields are required';
} elseif($password != $confirm_password) {
    $error = 'Passwords do not match';
} else {
    // Check if username already exists
    $query = 'SELECT * FROM users WHERE username = :username';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':username', $username);
    $stmt->execute();

    if($stmt->rowCount() > 0) {
        $error = 'Username already exists';
    } else {
        // Check if email already exists
        $query = 'SELECT * FROM users WHERE email = :email';
        $stmt = $conn->prepare($query);
        $stmt->bindParam(':email', $email);
        $stmt->execute();

        if($stmt->rowCount() > 0) {
            $error = 'Email already exists';
        } else {
            // Hash password
            $hashed_password = password_hash($password, PASSWORD_DEFAULT);

            // Insert new user
            $query = 'INSERT INTO users (username, password, email, first_name, last_name, user_type)
            | | | VALUES (:username, :password, :email, :first_name, :last_name, :user_type)';
            $stmt = $conn->prepare($query);

            $stmt->bindParam(':username', $username);
            $stmt->bindParam(':password', $hashed_password);
            $stmt->bindParam(':email', $email);
            $stmt->bindParam(':first_name', $first_name);
            $stmt->bindParam(':last_name', $last_name);
            $stmt->bindParam(':user_type', $user_type);

            if($stmt->execute()) {
                // Registration successful, redirect to login
                redirect('login.php?registered=1');
            } else {
                $error = 'Registration failed. Please try again.';
            }
        }
    }
}
```

Login:

First, the code looks up a user by their username in the database; then, after login, it checks how many notifications that user hasn't read yet and saves that number for the current session.

```
// Query to check user
$query = 'SELECT * FROM users WHERE username = :username';
$stmt = $conn->prepare($query);
$stmt->bindParam(':username', $username);
$stmt->execute();
```

```
// Get unread notifications count
$query = 'SELECT COUNT(*) as count FROM notifications WHERE user_id = :user_id AND is_read = 0';
$stmt = $conn->prepare($query);
$stmt->bindParam(':user_id', $user['user_id']);
$result = $stmt->fetch(PDO::FETCH_ASSOC);
$_SESSION['unread_count'] = $result['count'];
```

Mentorship:

This code is for handling the mentorship feature, which lets students request mentorship and mentors manage (accept/reject) those requests, sending relevant notifications to both parties.

- When a mentor accepts or rejects a mentorship request, the request's status is updated in the database and a notification is sent to the student.
- When a student submits a mentorship request, it is inserted into the database and a notification is sent to the selected mentor.

```
// Insert mentorship request
$query = 'INSERT INTO mentorship_requests (student_id, mentor_id, subject, message)
|   |   | VALUES (:student_id, :mentor_id, :subject, :message)';
$stmt = $conn->prepare($query);
$stmt->bindParam(':student_id', $user_id);
$stmt->bindParam(':mentor_id', $mentor_id);
$stmt->bindParam(':subject', $subject);
$stmt->bindParam(':message', $message);
$stmt->execute();
```

```

// Create notification for mentor
$title = 'New Mentorship Request';
$message = 'You have received a new mentorship request.';

$query = 'INSERT INTO notifications (user_id, type, title, message, related_id)
|   |   | VALUES (:user_id, "mentorship", :title, :message, :related_id)';
$stmt = $conn->prepare($query);
$stmt->bindParam(':user_id', $mentor_id);
$stmt->bindParam(':title', $title);
$stmt->bindParam(':message', $message);
$stmt->bindParam(':related_id', $request_id);

redirect('mentorship.php?requested=1');

```

```

// Handle mentorship actions
if(isset($_GET['action']) && isset($_GET['request_id']) && isMentor()) {
    $action = sanitize($_GET['action']);
    $request_id = (int)$_GET['request_id'];

    // Update mentorship request status
    $status = ($action == 'accept') ? 'accepted' : 'rejected';
    $query = 'UPDATE mentorship_requests SET status = :status WHERE request_id = :request_id AND mentor_id = :mentor_id';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':status', $status);
    $stmt->bindParam(':request_id', $request_id);
    $stmt->bindParam(':mentor_id', $user_id);
    $stmt->execute();

    // Create notification for student
    $query = 'SELECT student_id FROM mentorship_requests WHERE request_id = :request_id';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':request_id', $request_id);
    $stmt->execute();
    $result = $stmt->fetch(PDO::FETCH_ASSOC);
    $student_id = $result['student_id'];

    $title = ($action == 'accept') ? 'Mentorship Request Accepted' : 'Mentorship Request Rejected';
    $message = ($action == 'accept') ? 'Your mentorship request has been accepted.' : 'Your mentorship request has been rejected.';

    $query = 'INSERT INTO notifications (user_id, type, title, message, related_id)
|   |   | VALUES (:user_id, "mentorship", :title, :message, :related_id)';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':user_id', $student_id);
    $stmt->bindParam(':title', $title);
    $stmt->bindParam(':message', $message);
    $stmt->bindParam(':related_id', $request_id);

    redirect('mentorship.php');
}

```

```

// Get mentors for students
$mentors = [];
if(isStudent()) {
    $query = 'SELECT * FROM users WHERE user_type = "mentor" AND account_status = "active"';
    $stmt = $conn->prepare($query);
    $stmt->execute();
    $mentors = $stmt->fetchAll(PDO::FETCH_ASSOC);
}

// Get mentorship requests
if(isStudent()) {
    $query = 'SELECT mr.*, u.first_name, u.last_name
    FROM mentorship_requests mr
    JOIN users u ON mr.mentor_id = u.user_id
    WHERE mr.student_id = :student_id
    ORDER BY mr.created_at DESC';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':student_id', $user_id);
    $stmt->execute();
    $requests = $stmt->fetchAll(PDO::FETCH_ASSOC);
} else {
    $query = 'SELECT mr.*, u.first_name, u.last_name
    FROM mentorship_requests mr
    JOIN users u ON mr.student_id = u.user_id
    WHERE mr.mentor_id = :mentor_id
    ORDER BY mr.created_at DESC';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':mentor_id', $user_id);
    $stmt->execute();
    $requests = $stmt->fetchAll(PDO::FETCH_ASSOC);
}

```

Resources:

This code is for managing academic resources on the platform—allowing mentors to add new resources and users to filter or search existing ones.

- When a mentor posts a new resource, the code saves it to the database with all relevant details like title and category.
- It also retrieves resource categories and lists resources, applying search keywords and filters as needed.

```
// Insert new resource
$query = 'INSERT INTO resources (title, description, category_id, resource_type, link, created_by, is_public)
| | | VALUES (:title, :description, :category_id, :resource_type, :link, :created_by, 1)';
$stmt = $conn->prepare($query);
$stmt->bindParam(':title', $title);
$stmt->bindParam(':description', $description);
$stmt->bindParam(':category_id', $category_id);
$stmt->bindParam(':resource_type', $resource_type);
$stmt->bindParam(':link', $link);
$stmt->bindParam(':created_by', $_SESSION['user_id']);
```

```
// Get categories
$query = 'SELECT * FROM categories';
$stmt = $conn->prepare($query);
$stmt->execute();
$categories = $stmt->fetchAll(PDO::FETCH_ASSOC);

// Get resources with filtering
$category_id = isset($_GET['category']) ? (int)$_GET['category'] : 0;
$search = isset($_GET['search']) ? sanitize($_GET['search']) : '';

// Build the base query
$query = 'SELECT r.*, c.name as category_name, u.first_name, u.last_name
| | | FROM resources r
| | | JOIN categories c ON r.category_id = c.category_id
| | | LEFT JOIN users u ON r.created_by = u.user_id
| | | WHERE 1=1'; // Using 1=1 as a base condition

// Add filtering conditions
if($category_id > 0) {
|   $query .= ' AND r.category_id = :category_id';
}
if(!empty($search)) {
|   $query .= ' AND (r.title LIKE :search OR r.description LIKE :search)';
}

$query .= ' ORDER BY r.created_at DESC';
```

Announcement:

This code handles the announcements module for mentors and students, allowing creation, editing, deletion, per-user notifications, and filtering/display of announcements on the platform.

- It ensures necessary announcement-related tables and columns exist, lets mentors create, edit, or delete announcements, sends notifications to students, and displays filtered announcement lists along with category and author details.
- When a mentor edits or posts an announcement, the system updates the database and notifies students, while also supporting marking announcements as read/unread for each student.

```
// Handle POST actions (create, update, delete) for mentors
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isMentor()) {
    // Delete announcement
    if (!empty($_POST['delete_id'])) {
        $delete_id = (int)$_POST['delete_id'];
        $query = 'DELETE FROM announcements WHERE announcement_id = :id AND posted_by = :user_id';
        $stmt = $conn->prepare($query);
        $stmt->bindParam(':id', $delete_id);
        $stmt->bindParam(':user_id', $user_id);
        $stmt->execute();
        redirect('announcements.php?deleted=1');
    }
}
```

```
// Insert announcement
$query = 'INSERT INTO announcements (title, content, category_id, posted_by, priority, is_pinned, expires_at, notify_at)
VALUES (:title, :content, :category_id, :posted_by, :priority, :is_pinned, :expires_at, :notify_at)';
$stmt = $conn->prepare($query);
$stmt->bindParam(':title', $title);
$stmt->bindParam(':content', $content);
$stmt->bindParam(':category_id', $category_id);
$stmt->bindParam(':posted_by', $user_id);
$stmt->bindParam(':priority', $priority);
$stmt->bindParam(':is_pinned', $is_pinned);
$stmt->bindParam(':expires_at', $expires_at);
$stmt->bindParam(':notify_at', $notify_at);
$stmt->execute();

// Get all users to notify
$query = 'SELECT user_id FROM users WHERE user_type = "student"';
$stmt = $conn->prepare($query);
$stmt->execute();
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
// If no notify_at is set, notify immediately; otherwise students must opt-in and will be notified at notify_at
if (empty($notify_at)) {
    $notif_title = 'New Announcement: ' . htmlspecialchars($title);
    $message = 'A new announcement has been posted.';

    foreach ($students as $student) {
        $query = 'INSERT INTO notifications (user_id, type, title, message)
        | | | VALUES (:user_id, "announcement", :title, :message)';
        $stmt = $conn->prepare($query);
        $stmt->bindParam(':user_id', $student['user_id']);
        $stmt->bindParam(':title', $notif_title);
        $stmt->bindParam(':message', $message);
        $stmt->execute();
    }
}
```

```
// If mentor requested to edit, load announcement into edit mode
if (isMentor() && isset($_GET['action']) && $_GET['action'] === 'edit' && !empty($_GET['id'])) {
    $edit_id = (int)$_GET['id'];
    $query = 'SELECT * FROM announcements WHERE announcement_id = :id AND posted_by = :user_id';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':id', $edit_id);
    $stmt->bindParam(':user_id', $user_id);
    $stmt->execute();
    $edit_announcement = $stmt->fetch(PDO::FETCH_ASSOC);
    if ($edit_announcement) {
        $edit_mode = true;
    }
}
```

```
// Get announcements - FIXED QUERY
$query = 'SELECT a.*, c.name as category_name, u.first_name, u.last_name
FROM announcements a
JOIN categories c ON a.category_id = c.category_id
JOIN users u ON a.posted_by = u.user_id
WHERE (a.expires_at IS NULL OR a.expires_at > NOW())
ORDER BY a.is_pinned DESC, a.priority DESC, a.created_at DESC'; // Changed posted_at to created_at
$stmt = $conn->prepare($query);
$stmt->execute();
$announcements = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
{
$conn->exec("CREATE TABLE IF NOT EXISTS announcement_reads (
    id INT AUTO_INCREMENT PRIMARY KEY,
    announcement_id INT NOT NULL,
    user_id INT NOT NULL,
    read_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY (announcement_id, user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");
try (Exception $e) {
    // ignore

    add notify_at column to announcements if it doesn't exist
    {
        $colCheck = $conn->prepare("SELECT COUNT(*) as cnt FROM information_schema.COLUMNS WHERE
        TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'announcements' AND COLUMN_NAME = 'notify_at'");
        $colCheck->execute();
        $hasCol = $colCheck->fetch(PDO::FETCH_ASSOC);
        if (empty($hasCol) || $hasCol['cnt'] == 0) {
            $conn->exec("ALTER TABLE announcements ADD COLUMN notify_at DATETIME NULL AFTER expires_at");
        }
    }
}
```


Jobs:

This code is for the jobs module, enabling mentors to post new jobs, view applicants for their jobs, and allowing users to filter job listings on the platform.

- When a mentor views the applicants for a job they posted, the code checks job ownership and then retrieves a list of students who have applied to that job.
- The code also allows mentors to insert new job entries and enables all users to filter and display available jobs based on type or mode.

```
// Handle edit request (mentor only) - load job into form for editing
if (isset($_GET['edit']) && isMentor()) {
    $edit_id = (int)$_GET['edit'];
    $q = 'SELECT * FROM jobs WHERE job_id = :job_id';
    $s = $conn->prepare($q);
    $s->bindParam(':job_id', $edit_id);
    $s->execute();
    $job_to_edit = $s->fetch(PDO::FETCH_ASSOC);
    if ($job_to_edit && $job_to_edit['posted_by'] == $user_id) {
        $edit_mode = true;
        $edit_job = $job_to_edit;
    } else {
        // Not found or not owner - redirect with error flag
        redirect('jobs.php?edit=0');
    }
}
```

```
// Handle applicants view (mentor only)
if (isset($_GET['applicants']) && isMentor()) {
    $app_job_id = (int)$_GET['applicants'];
    $q = 'SELECT * FROM jobs WHERE job_id = :job_id';
    $s = $conn->prepare($q);
    $s->bindParam(':job_id', $app_job_id);
    $s->execute();
    $job_row = $s->fetch(PDO::FETCH_ASSOC);
    if ($job_row && $job_row['posted_by'] == $user_id) {
        $show_applicants = true;
        $applicants_job = $job_row;

        $q2 = 'SELECT ja.student_id, u.first_name, u.last_name, u.email, ja.applied_at
        FROM job_applications ja JOIN users u ON ja.student_id = u.user_id WHERE ja.job_id = :job_id ORDER BY ja.applied_at DESC';
        $s2 = $conn->prepare($q2);
        $s2->bindParam(':job_id', $app_job_id);
        $s2->execute();
        $applicants_list = $s2->fetchAll(PDO::FETCH_ASSOC);
    } else {
        // Not owner or not found
        redirect('jobs.php?applicants=0');
    }
}
```

```
// Check if already applied
$query = 'SELECT * FROM job_applications WHERE job_id = :job_id AND student_id = :student_id';
$stmt = $conn->prepare($query);
$stmt->bindParam(':job_id', $job_id);
$stmt->bindParam(':student_id', $user_id);
$stmt->execute();

if ($stmt->rowCount() == 0) {
    // Insert application
    $query = 'INSERT INTO job_applications (job_id, student_id) VALUES (:job_id, :student_id)';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':job_id', $job_id);
    $stmt->bindParam(':student_id', $user_id);
    $stmt->execute();

    // Get job details
    $query = 'SELECT * FROM jobs WHERE job_id = :job_id';
    $stmt = $conn->prepare($query);
    $stmt->bindParam(':job_id', $job_id);
    $stmt->execute();
    $job = $stmt->fetch(PDO::FETCH_ASSOC);
}
```

```
// Insert job
$query = 'INSERT INTO jobs (title, description, company, location, type, mode, salary_min, salary_max, posted_by, deadline)
VALUES (:title, :description, :company, :location, :type, :mode, :salary_min, :salary_max, :posted_by, :deadline)';
$stmt = $conn->prepare($query);
$stmt->bindParam(':title', $title);
$stmt->bindParam(':description', $description);
$stmt->bindParam(':company', $company);
$stmt->bindParam(':location', $location);
$stmt->bindParam(':type', $type);
$stmt->bindParam(':mode', $mode);
$stmt->bindParam(':salary_min', $salary_min);
$stmt->bindParam(':salary_max', $salary_max);
$stmt->bindParam(':posted_by', $user_id);
$stmt->bindParam(':deadline', $deadline);
$stmt->execute();
```

```
// Get jobs with filter @var mixed
$type = isset($_GET['type']) ? sanitize($_GET['type']) : '';
$mode = isset($_GET['mode']) ? sanitize($_GET['mode']) : '';

$query = 'SELECT j.*, u.first_name, u.last_name
FROM jobs j
LEFT JOIN users u ON j.posted_by = u.user_id
WHERE j.is_active = 1';

if (!empty($type)) {
    $query .= ' AND j.type = :type';
}
}
```

Profile:

This code is for managing user profiles, allowing users to update their personal information and view their profile details on the platform.

- When a user updates their profile, the code saves the new details in the database.
- To display user information, it retrieves the user's data from the database using their user ID.

```
// Update profile
$query = 'UPDATE users SET
    first_name = :first_name,
    last_name = :last_name,
    email = :email,
    bio = :bio,
    phone = :phone,
    department = :department,
    graduation_year = :graduation_year,
    expertise = :expertise,
    linkedin_profile = :linkedin_profile
WHERE user_id = :user_id';
```

```
// Get user data
$query = 'SELECT * FROM users WHERE user_id = :user_id';
$stmt = $conn->prepare($query);
$stmt->bindParam(':user_id', $user_id);
$stmt->execute();
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

Source Code Repository

<https://drive.google.com/drive/folders/1L5qtn2IdOm93UDgjdOTOa9sqxyQ5xHZY?usp=sharing>

Conclusion

The BSC Support Hub serves as an essential tool for undergraduate students, offering them a centralized platform for academic support, career guidance, and personal development. By providing easy access to resources, mentorship, job opportunities, and important announcements, it ensures that students have everything they need to succeed. The personalized features, like the dashboard help students stay organized and on track with their goals. This platform fosters a student-first approach, promoting collaboration, growth, and well-being. Overall, the BSC Support Hub is a valuable resource that enhances the student experience, empowering them to achieve their academic and career aspirations.

References

W3Schools. Php tutorial: <https://www.w3schools.com/php/default.asp>

BroCode PHP tutorial: <https://youtu.be/zZ6vybT1HQs>

Hostinger Html and css:

https://www.hostinger.com/tutorials/how-to-make-a-website?utm_campaign=Generic-Tutorials-DSA|NT:Se|LO:BD&utm_medium=ppc&gad_source=1&gad_campaignid=20361099779&gbraid=0AAAAADMy-haf8QjCyGzQBAHYBnSgN2h6B&gclid=CjwKCAjwz_bABhAGEiwAm-P8YYyRe6wF-_SRFQKcE6aks34664NKf3aIVPFMNk3JEcN7zUI4Bw6STBoCqUUQAvD_BwE

TutorialsPoint. PHP tutorial: <https://www.tutorialspoint.com/php/>

PHP & MySQL by Faisal Bin Ashraf sir :

https://www.youtube.com/playlist?list=PLJW6cU20q-SOQB1Dqj2vF9mcNqgh4-z_j